

SudhaarAI - Generative Code Assistant and Context-Based Code Optimizer



Department of Computer Science

Raqib Hayat NUM-BSCS-2022-40

Abu Bakar NUM-BSCS-2022-41

2026

Final year project report submitted in partial fulfillment of the requirement for the degree of
Bachelor of Science in Computer Science

**Namal University,
30-KM, Talagang Road, Mianwali, Pakistan.**

www.namal.edu.pk

DECLARATION

The project report titled **SudhaarAI - Generative Code Assistant and Context-Based Code Optimizer** is submitted in partial fulfillment of the degree of Bachelors of Science in Computer Science, to the Department of Computer Science at Namal University, Mianwali, Pakistan.

It is declared that this is an original work done by the team members listed below, under the guidance of our supervisor **Mr. Shahzad Arif**. No part of this project and its report is plagiarized from anywhere, and any help taken from previous work is cited properly.

No part of the work reported here is submitted in fulfillment of requirement for any other degree/ qualification in any institute of learning.

Team Members	University ID	Signatures
Raqib Hayat	40	_____
Abu Bakar	41	_____

Supervisor

Mr. Shahzad Arif

Signatures with date

Contents

DECLARATION	1
Abstract	5
1 Chapter 1: Introduction	6
1.1 Purpose	6
1.2 Scope	7
1.3 Definitions, Acronyms, And Abbreviations	10
1.4 References	11
1.5 Overview	12
2 Chapter 2: System Description	13
2.1 Code Quality Assessment Framework	14
2.2 Overall Description	14
2.3 System Environment	16
2.4 Product Perspective	18
2.5 Product Features	20
3 Chapter 3: Analysis And Design	23
3.1 Functional Requirements	23
3.2 Non-Functional Requirements	27
3.3 System Models	28
3.3.1 Data Flow Diagrams (DFD)	28
3.3.2 Use Case Diagrams	30
3.3.3 ER Diagram	31
3.3.4 Sequence And Activity Diagrams	32
3.4 External Interface Requirements	33
3.4.1 User Interfaces	33
3.4.2 Hardware Interfaces	36
3.4.3 Software Interfaces	36
3.4.4 Communication Interfaces	36
3.5 Constraints And Limitations	37
3.6 Assumptions And Dependencies	38

4	Chapter 4: Remaining Work And Timeline	39
4.1	Development Methodology And Architecture	39
4.2	Current Implementation Status (Phase 1)	40
4.3	Remaining Work (Phase 2 - Next Semester)	40
4.3.1	Individual Task Distribution	41
4.4	Success Criteria And Evaluation	42
5	Technology Stack Implementation	43
	References	45
	Appendices	46
6	Appendix A: Project Dependencies And Setup	46
6.1	A.1 Project Dependencies	46
6.1.1	Frontend Dependencies (package.json)	46
6.1.2	Backend Dependencies (requirements.txt)	47
6.2	A.2 Installation And Setup	47
6.2.1	Frontend Setup	47
6.2.2	Backend Setup	48
6.2.3	System Requirements	48
7	Appendix B: System Overview And Validation	49
7.1	B.1 System Architecture	49
7.2	B.2 System Validation	50
8	Appendix C: Dataset Statistics	51
8.1	C.1 Knowledge Base Summary	51
8.2	C.2 Processing Pipeline Results	51
8.3	C.3 Content Coverage By Category	51
9	Appendix D: Future Work	51
9.1	D.1 Planned Features (Phase 2)	51
9.2	D.2 Research Directions	52
10	Appendix E: Glossary	52

List of Figures

1	<i>Data Flow Diagram Level 0</i>	29
2	<i>Data Flow Diagram Level 1</i>	29
3	<i>Use Case Diagram</i>	30
4	<i>Entity Relationship Diagram</i>	31
5	<i>Sequence Diagram</i>	32
6	<i>Main Application Interface</i>	34
7	<i>Analysis Results Dashboard</i>	34
8	<i>AI Expert Recommendations</i>	35
9	<i>Generated Code View</i>	35
10	<i>Turnitin Report</i>	53

List of Tables

1	<i>Phase 1 Completed Work</i>	40
2	<i>Phase 2 Planned Work</i>	40
3	<i>Outstanding Tasks With Status And Deadlines</i>	41
4	<i>Abu Bakar Task Timeline</i>	41
5	<i>Raqib Hayat Task Timeline</i>	42
6	<i>Collaborative Timeline</i>	42
7	<i>Frontend Test Suite Results</i>	50
8	<i>Retrieval System Performance Metrics</i>	50
9	<i>Dataset Collection Statistics</i>	51
10	<i>Phase 1 And Phase 2 Processing Metrics</i>	51
11	<i>Best Practice Category Coverage</i>	52

Abstract

The rapid advancement of generative artificial intelligence has revolutionized software development, yet existing tools like GitHub Copilot and ChatGPT prioritize functionality over optimization, producing code that works but may lack efficiency. This project develops SudhaarAI, an intelligent web-based Python code analysis and optimization system that addresses this gap through a novel two-stage architecture combining client-side metrics calculation with server-side Retrieval-Augmented Generation powered recommendations. The system employs a custom client-side analysis engine to calculate code metrics including loop counting, conditional analysis, and basic complexity estimation without server communication, ensuring privacy and immediate feedback. The backend leverages a curated offline knowledge base containing Python best practices from official Python Enhancement Proposals, community tutorials, and documentation, stored in ChromaDB vector database with embeddings for semantic retrieval using Retrieval-Augmented Generation. The system integrates open-source Large Language Models through Ollama to generate citation-backed optimization suggestions grounded in authoritative sources. A key innovation is the Context-Aware Semantic Text chunking feature that optimizes how code and documentation are segmented for Retrieval-Augmented Generation processing, demonstrating improved retrieval quality over traditional fixed-size chunking. The current implementation provides Python programmers at any level, from junior to senior, with immediate code quality assessment through Monaco Editor integration, comprehensive results visualization with score gauges and detailed analysis tabs, and intelligent query processing with intent classification and document reranking. Future development focuses on creating a fully web-based platform that delivers complete code optimization functionalities to Python programmers at any skill level, enabling them to write better, more efficient, and maintainable code. The system provides real-time analysis and context-aware suggestions that improve code quality, maintainability, and adherence to industry standards while maintaining privacy through local processing and offline knowledge base operation.

Chapter 1: Introduction

SudhaarAI is an intelligent web-based Python code analysis and optimization system that addresses the growing need for code quality improvement in software development. While existing AI code generation tools like GitHub Copilot and ChatGPT demonstrate impressive capabilities, they often prioritize functionality over optimization, producing code that works but may lack efficiency in computational resources, execution time, or adherence to best practices.

The system employs a novel two-stage architecture combining client-side code analysis using a custom JavaScript-based Python parser for real-time metrics calculation with server-side Retrieval-Augmented Generation (RAG) for knowledge-enhanced recommendations. The frontend calculates basic code complexity metrics and structural analysis without server communication, ensuring privacy and immediate feedback. The backend RAG system retrieves relevant context from a curated offline knowledge base containing Python Enhancement Proposals (PEPs), community tutorials, and official documentation, then generates contextually-aware optimization suggestions through open-source Large Language Models via Ollama.

A key innovation is the Context-Aware Semantic Text (CAST) chunking feature, which optimizes how code and documentation are segmented for RAG processing, demonstrating improved retrieval quality over traditional fixed-size chunking strategies.

The goal is to provide Python programmers at any level, from junior developers learning best practices to senior professionals optimizing production code, with an intelligent assistant that delivers real-time code analysis and context-aware optimization suggestions grounded in authoritative sources, enabling them to write better, more efficient, and maintainable code.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to clearly define the functional and non-functional requirements for the AI-Powered Code Optimization Assistant. It serves as a blueprint for the development team, ensuring that all stakeholders

have a shared understanding of what is to be built.

In an industrial context, this document guides the transition from a research-based prototype to a robust software tool. It outlines the specific features needed to support developers in a professional environment, detailing how the system interacts with users, processes data, and maintains high standards of code quality.

1.2 Scope

This project focuses on the development of an intelligent web-based code analysis and optimization system specifically designed for Python development.

Current Implementation (Phase 1 - 40% Complete):

- To create a web-based application that provides real-time Python code analysis and optimization suggestions.
- To implement a two-stage analysis pipeline combining client-side metrics calculation with server-side RAG-powered recommendations.
- To develop an offline RAG knowledge base containing curated Python best practices from authoritative sources (PEPs, official documentation, community tutorials).
- To provide Context-Aware Semantic Text (CAST) chunking capabilities for optimizing RAG retrieval quality.
- To integrate Ollama for generating contextually-aware optimization suggestions using open-source LLMs.
- To deliver immediate code quality feedback without requiring server communication for basic metrics.

Next Semester Development (Phase 2 - Remaining FYP Work):

- To add natural language code generation capabilities alongside existing optimization features.

- To implement user authentication and session management systems.
- To deploy the system on cloud infrastructure (Azure or similar) for broader accessibility.
- To enhance the web-based interface with additional features for improved user experience.

Key System Features:

- **Client-Side Code Analysis:** Custom JavaScript-based Python parser for calculating loop counts, conditional analysis, nesting depth, and basic complexity metrics without server communication.
- **Monaco Code Editor:** Custom code editor with syntax highlighting, drag-and-drop file upload, and adjustable height.
- **RAG-Powered Optimization:** Server-side Retrieval-Augmented Generation system that retrieves relevant best practices from curated knowledge base and generates contextual recommendations.
- **CAST Chunking Tool:** Context-Aware Semantic Text chunking feature allowing users to compare different chunking strategies and optimize RAG prompt generation.
- **Offline Knowledge Base:** Curated collection of Python best practices, design patterns, and optimization strategies from PEPs, Real Python, Python documentation, and Stack Overflow.
- **Open-Source LLM Integration:** Support for local Ollama models for privacy-preserving AI assistance.
- **Real-Time Feedback:** Immediate code quality metrics and optimization suggestions with visual representations (gauges, charts, detailed breakdowns).

AI-Powered Functionalities:

- **Two-Stage Analysis Pipeline:** Stage 1 (Frontend) calculates code metrics using custom JavaScript parser; Stage 2 (Backend) provides RAG-enhanced optimization suggestions using Ollama LLMs.

- **Query Processing:** Intent classification, query routing, and query rewriting to optimize retrieval from the knowledge base.
- **Context Assembly:** Document reranking, diversity filtering, and citation tracking to ensure high-quality, relevant recommendations.
- **Semantic Retrieval:** Hybrid search combining vector similarity and keyword matching for optimal context retrieval.
- **Citation-Backed Recommendations:** All optimization suggestions include citations to authoritative sources (PEPs, official docs, tutorials).
- **Multi-Turn Conversations:** Session management supporting iterative refinement of code optimization queries.

Supported Programming Language:

- **Python:** Exclusive focus on Python programming language throughout the project

Web-Based Application Benefits:

- **Immediate Accessibility:** No installation required; accessible from any modern web browser.
- **Client-Side Analysis:** Core metrics calculation runs in browser using custom JavaScript parser, ensuring privacy and reducing server load.
- **Offline RAG Knowledge:** All best practices stored locally in vector database, eliminating dependency on external APIs.
- **Cross-Platform Compatibility:** Works on Windows, macOS, and Linux without platform-specific builds.
- **Rapid Iteration:** Web-based architecture enables quick updates and feature additions.

1.3 Definitions, Acronyms, And Abbreviations

- **API (Application Programming Interface):** A set of rules that allows different software applications to communicate with each other.
- **AST (Abstract Syntax Tree):** A tree representation of the abstract syntactic structure of source code written in a programming language.
- **Big-O Notation:** Mathematical notation used to describe the computational complexity of algorithms in terms of time and space requirements.
- **ChromaDB:** An open-source vector database designed for storing and retrieving high-dimensional embeddings.
- **Code Generation:** The process of automatically creating source code from high-level specifications or natural language descriptions.
- **Code Optimization:** The process of modifying software to improve efficiency, reduce resource consumption, or enhance performance.
- **Conversational AI:** AI systems designed to engage in natural language conversations with users through text or speech.
- **FastAPI:** A modern, high-performance web framework for building APIs with Python based on standard Python type hints.
- **Intent Classification:** NLP technique for determining the purpose or goal behind a user's query or statement.
- **LLM (Large Language Model):** Deep learning models trained on vast amounts of text data to understand and generate human-like text.
- **NLP (Natural Language Processing):** Branch of AI focused on enabling computers to understand, interpret, and generate human language.
- **RAG (Retrieval-Augmented Generation):** AI technique that enhances language model responses by retrieving relevant information from external knowledge bases.

- **Semantic Search:** Search technique that understands the meaning and context of queries rather than just matching keywords.
- **Vector Embeddings:** Numerical representations of text, code, or other data in high-dimensional space that capture semantic meaning.

1.4 References

- [1] GitHub, "GitHub Copilot: Your AI pair programmer," GitHub. [Online]. Available: <https://github.com/features/copilot>.
- [2] Cursor, "Cursor AI: The AI-first code editor," Cursor. [Online]. Available: <https://cursor.sh/>.
- [3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.
- [4] H. Shi et al., "RAG Applications in Code Generation: Improving Quality through Semantic Retrieval," in *Proc. ICSE*, 2024.
- [5] "ChromaDB Documentation," Chroma. [Online]. Available: <https://docs.trychroma.com/>.
- [6] "Sentence-Transformers Documentation," SBERT.net. [Online]. Available: <https://www.sbert.net/>.
- [7] H. Massalin, "Superoptimizer: A Look at the Smallest Program," in *Proc. ASPLOS*, 1987.
- [8] A. Palit and R. Sharma, "Reinforcement Learning for Code Refactoring," in *Proc. ICML*, 2024.
- [9] S. Maini et al., "HSHEP: Optimization-based Code Smell Refactoring Sequencing," in *IEEE Trans. Software Eng.*, 2024.
- [10] "FastAPI Documentation," FastAPI. [Online]. Available: <https://fastapi.tiangolo.com/>.

1.5 Overview

This SRS document is organized into the following sections:

- **Section 1: Introduction** sets the context, defining the problem, scope, and key terms.
- **Section 2: System Description** describes the product context, user characteristics, limitations, and features.
- **Section 3: Analysis and Design** details the functional requirements, system models (DFD, ERD, Use Cases), and interface specifications.
- **Section 4: Remaining Work and Timeline** outlines the outstanding tasks and the project schedule.
- **References** lists all citations used in the document.
- **Appendices** provide additional technical details such as system configurations.

Chapter 2: System Description

Background And Literature Review

The domain of software quality assurance has traditionally relied on static analysis tools. Systems like Pylint and Flake8 utilize Abstract Syntax Trees (AST) to enforce syntactic correctness and stylistic consistency based on predefined rules. While effective for syntax errors, these tools often lack the semantic understanding required to offer deeper, logic-based optimization suggestions.

Recent advancements in Large Language Models (LLMs) have revolutionized code generation and analysis. These models can understand intent and generate complex code solutions. However, a significant limitation identified in the literature is "hallucination," where models generate plausible but incorrect or non-existent code patterns. Furthermore, general-purpose LLMs may not always align with the specific, evolving standards of a particular language's ecosystem, such as the latest Python Enhancement Proposals (PEPs).

To mitigate these limitations, the concept of Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. RAG combines the generative capabilities of LLMs with the precise retrieval of information from an external knowledge base. By grounding the model's responses in retrieved, authoritative documents, RAG significantly reduces hallucinations and ensures relevance.

This project implements a RAG-based architecture specifically optimized for Python code analysis. Unlike standard code completion tools (e.g., GitHub Copilot) which rely primarily on training data, this system dynamically retrieves context from a curated vector database of Python best practices. This approach aligns with recent trends in "Context-Aware Coding Assistants," ensuring that recommendations are not just syntactically correct but arguably "best practice" citations from trusted sources.

2.1 Code Quality Assessment Framework

The system incorporates a code quality assessment framework to evaluate Python code and determine optimization priorities through client-side analysis using a custom JavaScript-based Python parser.

Current Metrics Calculation:

The frontend analysis engine calculates the following metrics:

- **Loop Analysis:** Count of for loops, while loops, and maximum nesting depth.
- **Conditional Analysis:** Count of if statements and conditional branches.
- **Arithmetic Operations:** Count and location of mathematical operations.
- **Variable Analysis:** Count of unique variables and variable names.
- **Basic Complexity Estimation:** Simplified time and space complexity analysis.

These metrics are combined to generate an overall optimization score (0-10) that indicates code quality and identifies areas requiring improvement.

2.2 Overall Description

SudhaarAI is designed as an intelligent web-based code analysis and optimization system that augments Python developer capabilities through a unique two-stage architecture combining client-side metrics calculation with server-side RAG-powered recommendations.

User Characteristics:

- **Primary Users - Programmers:** Python programmers of any level, from students learning programming fundamentals to professional developers, who want to improve code quality, maintainability, and adherence to best practices.

- **Secondary Users - Educators:** Instructors using the tool to demonstrate code quality concepts and Python best practices to students.
- **Secondary Users - Code Reviewers:** Technical leads and senior developers using the tool to assist in code review processes.
- **Expertise Range:** Users range from beginners learning Python fundamentals to experienced developers seeking advanced optimization techniques.
- **Technical Requirements:** Users need basic understanding of Python programming; no specialized knowledge of RAG or LLMs required.

Operational Modes:

- **Code Analysis Mode (Current):** Users submit Python code for immediate metrics calculation and receive detailed quality assessment with visual feedback.
- **RAG Optimization Mode (Current):** Users query the system for best practice recommendations, receiving citation-backed suggestions from curated knowledge base.
- **CAST Chunking Mode (Current):** Users experiment with different text chunking strategies to optimize RAG prompt generation and context retrieval.
- **Code Generation Mode (Planned):** Users will provide natural language descriptions to generate optimized Python code implementations.

System Constraints:

- **Language Support:** Currently limited to Python.
- **Browser Compatibility:** Requires modern browser with JavaScript ES6+ support for custom analysis engine.
- **Code Size Limits:** Client-side analysis optimal for files under 1000 lines; larger files may experience performance degradation.
- **LLM Response Time:** Backend RAG queries depend on local LLM inference speed (10-60 seconds typical).

- **Knowledge Base Currency:** Offline knowledge base requires periodic updates to include latest Python best practices.
- **Network Dependency:** Backend RAG features require server connectivity; frontend analysis works offline.
- **Deployment:** Currently local development only; cloud deployment planned for future phases.

Assumptions and Dependencies:

- **Assumption:** Users have access to modern web browsers supporting WebAssembly and ES6+ JavaScript.
- **Assumption:** Users are working with Python code and seek to improve code quality and adherence to best practices.
- **Assumption:** The curated knowledge base contains representative and authoritative Python best practices.
- **Dependency:** Custom JavaScript parser for client-side code analysis.
- **Dependency:** Ollama for local open-source LLM inference; system requires functional Ollama installation.
- **Dependency:** ChromaDB vector database for semantic search and retrieval.
- **Dependency:** sentence-transformers library for generating embeddings.
- **Dependency:** Authentication system for user management and session persistence.

2.3 System Environment

Client-Side Requirements (Frontend):

- **Web Browser:** Modern browser with JavaScript ES6+ support (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+).

- **Memory:** Minimum 4 GB RAM; 8 GB recommended for optimal performance.
- **Display:** Minimum 1366x768 resolution; 1920x1080 or higher recommended for optimal user experience.
- **Network:** Internet connection required for initial load and backend RAG queries; frontend analysis works offline after initial load.
- **JavaScript:** Must be enabled for application functionality.

Server-Side Requirements (Backend):

- **Operating System:** Windows 10/11, macOS 12+, or Linux (Ubuntu 20.04+) for development server.
- **Processor:** Minimum quad-core CPU; 8+ cores recommended for LLM inference.
- **Memory:** Minimum 16 GB RAM; 32 GB recommended for running local LLM models.
- **Storage:** At least 10 GB free disk space for knowledge base, vector database, and LLM models.
- **GPU (Optional):** CUDA-compatible GPU with 8+ GB VRAM for accelerated LLM inference.

Software Stack:

- **Frontend Framework:** React 18+ with Vite build system.
- **Code Editor:** Custom code editor for browser-based code editing with syntax highlighting.
- **Client-Side Analysis:** Custom JavaScript parser for code analysis and metrics calculation.
- **Backend Framework:** Flask/FastAPI for REST API and RAG pipeline orchestration.
- **Python Version:** Python 3.10+ for backend services.

- **Vector Database:** ChromaDB for storing and querying embeddings.
- **Embedding Model:** sentence-transformers (all-MiniLM-L6-v2) for semantic search.
- **LLM Providers:** Ollama (local inference) for open-source model support.
- **Package Management:** uv for Python dependency management (replaces pip/poetry).

Development Tools:

- **Frontend:** Node.js 18+, npm/pnpm, ESLint, Prettier.
- **Backend:** Python 3.10+, uv package manager.
- **Code Analysis:** Custom JavaScript parser for Python code analysis and basic complexity metrics.
- **Version Control:** Git for source code management.

Deployment Environment:

- **Current:** Local development environment only.
- **Planned:** Cloud deployment on Azure or similar platform.
- **Frontend Hosting:** Static site hosting (Vercel, Netlify, or Azure Static Web Apps).
- **Backend Hosting:** Container-based deployment (Azure Container Instances or App Service).
- **Database:** Managed vector database service or self-hosted ChromaDB instance.

2.4 Product Perspective

SudhaarAI functions as an **web-based code analysis and optimization system** with a unique two-stage architecture combining client-side analysis and server-side RAG-powered recommendations.

- **Relationship to Existing Tools:**

- Unlike **Generic AI Chatbots (ChatGPT, Claude)**: SudhaarAI provides specialized Python code analysis with citations to authoritative sources and grounded recommendations based on curated best practices.
- Unlike **Code Analysis Tools (Pylint, Flake8)**: It combines static analysis with AI-powered contextual recommendations, explaining not just what to fix but why and how.
- Unlike **AI Code Assistants (GitHub Copilot)**: It focuses on optimization and best practices rather than just code completion, with transparent citations to source materials.
- Unlike **Traditional RAG Systems**: It features a novel two-stage architecture with client-side metrics calculation and CAST chunking for optimized context retrieval.

- **Unique Value Propositions:**

- **Privacy-First Design**: Client-side analysis using custom JavaScript parser means code never leaves the browser for basic metrics.
- **Offline Knowledge Base**: All best practices stored locally; no dependency on external APIs or internet connectivity for knowledge retrieval.
- **Citation-Backed Recommendations**: Every suggestion includes references to PEPs, official documentation, or community tutorials.
- **Open-Source LLMs**: Uses Ollama for local inference instead of proprietary APIs, ensuring transparency and cost-effectiveness.
- **CAST Chunking Innovation**: Allows users to experiment with and optimize RAG retrieval strategies through context-aware semantic chunking.

- **Target Market Position:**

- Serves Python developers seeking to improve code quality and learn best practices.
- Provides an educational tool for computer science students learning Python development.

- Offers professional developers a privacy-respecting alternative to cloud-based AI coding assistants.
 - Positions as a research platform for exploring RAG optimization techniques in code analysis.
- **Future Evolution:**
 - Addition of natural language code generation alongside current optimization capabilities.
 - Potential cloud deployment for broader accessibility while maintaining privacy-first principles.

2.5 Product Features

SudhaarAI provides a comprehensive suite of code analysis and optimization capabilities through a two-stage architecture:

1. Client-Side Code Analysis (Stage 1 - Frontend)

- Real-time Python code analysis in browser using custom JavaScript parser.
- Loop counting and analysis (for loops, while loops, nesting depth).
- Conditional analysis (if statements and branches).
- Arithmetic operation counting and location tracking.
- Variable analysis and unique variable counting.
- Basic complexity estimation (simplified time/space complexity).
- Immediate feedback without server communication, ensuring privacy.
- Visual representations: score gauges, feature cards, complexity charts.

2. RAG-Powered Optimization (Stage 2 - Backend)

- Intelligent query processing with intent classification and routing.
- Semantic retrieval from curated offline knowledge base.

- Hybrid search combining vector similarity (70%) and keyword matching (30%).
- Document reranking with diversity filtering for optimal context selection.
- Context assembly with citation tracking and source attribution.
- LLM-powered response generation using Ollama models.
- Citation-backed recommendations referencing PEPs, official docs, and tutorials.
- Multi-turn conversation support with session management.

3. Context-Aware Semantic Text (CAST) Chunking

- Interactive tool for comparing chunking strategies (context-aware vs fixed-size).
- Real-time visualization of chunk boundaries and overlap.
- Automatic RAG prompt generation from chunked code.
- Statistical comparison of chunking effectiveness.
- Export functionality for chunks and prompts.
- Configurable chunk size and overlap parameters.
- Demonstration of RAG optimization techniques.

4. Curated Knowledge Base

- 500+ Python best practices from authoritative sources.
- Tier 1: Official Python Enhancement Proposals (PEPs).
- Tier 2: Community tutorials (Real Python, Python Guide, GeeksforGeeks).
- Tier 3: Python official documentation and curated Stack Overflow Q&A.
- 2,319 embeddings (909 text chunks + 1,410 code blocks).
- ChromaDB vector database with three specialized collections.
- Offline operation with no external API dependencies.
- Regular updates with latest Python best practices.

5. Open-Source LLM Integration

- Primary: Ollama for local LLM inference (privacy-preserving).
- Support for multiple open-source models (Qwen, gpt-oss, LLama, etc.).

- Configurable temperature and token limits.
- Automatic model selection based on query complexity.
- No dependency on proprietary APIs, using only open-source models.
- Cost-effective operation with local models.

6. Professional Code Editor

- Custom code editor integration for code editing experience.
- Python syntax highlighting and basic auto-completion.
- Code folding and bracket matching.
- Drag-and-drop file upload support.
- Adjustable editor height for user preference.
- Example code loading for quick testing.
- Copy and download functionality for results.

7. Comprehensive Results Visualization

- Overall optimization score with color-coded gauge.
- Individual metric cards (complexity, maintainability, efficiency, readability).
- Detailed tabbed interface (Overview, Methodology, Detailed Analysis, RAG Prompt).
- Cost breakdown visualization for optimization priorities.
- Correlation analysis between metrics.
- Modal dialogs for in-depth metric explanations.
- Export functionality (JSON format with metadata).

8. Session and History Management

- Conversation session tracking for multi-turn interactions.
- Query history with context preservation.
- Session timeout management (configurable).
- Future: User authentication and persistent history across devices.

Chapter 3: Analysis And Design

This section details the functional and non-functional requirements, system models, and interface specifications for the AI-Powered Code Optimization Assistant. It provides a blueprint for the system's architecture and logic.

3.1 Functional Requirements

The system's functional requirements are organized by core capabilities and implementation phases:

Phase 1 - Current Implementation:

- **FR-01: Client-Side Code Analysis**

- The system shall analyze Python code in the browser using custom JavaScript parser.
- The system shall count loops (for loops, while loops) and calculate maximum nesting depth.
- The system shall count conditional statements (if statements) and branches.
- The system shall count arithmetic operations and track their locations.
- The system shall count unique variables and extract variable names.
- The system shall estimate basic time and space complexity.
- The system shall generate an overall optimization score (0-100) based on calculated metrics.
- The system shall provide immediate feedback without requiring server communication.

- **FR-02: Code Editor Interface**

- The system shall provide custom code editor for code input with Python syntax highlighting.

- The system shall support drag-and-drop file upload for Python files.
- The system shall validate file types and size limits (maximum 1 MB).
- The system shall allow adjustable editor height for user preference.
- The system shall provide example code loading for quick testing.
- The system shall support code clearing and reset functionality.

- **FR-03: Results Visualization**

- The system shall display overall optimization score with color-coded gauge (red/yellow/green).
- The system shall show individual metric cards for complexity, maintainability, efficiency, and readability.
- The system shall provide tabbed interface for Overview, Methodology, Detailed Analysis, and RAG Prompt views.
- The system shall visualize cost breakdown for optimization priorities.
- The system shall display correlation analysis between metrics.
- The system shall provide modal dialogs for detailed metric explanations.

- **FR-04: RAG Knowledge Base**

- The system shall maintain an offline knowledge base of 500+ Python best practices.
- The system shall organize knowledge in three tiers: PEPs (Tier 1), community tutorials (Tier 2), and documentation/Q&A (Tier 3).
- The system shall store 2,319 embeddings (909 text chunks + 1,410 code blocks) in ChromaDB.
- The system shall support semantic search using sentence-transformers embeddings.
- The system shall implement hybrid search combining vector similarity (70%) and keyword matching (30%).
- The system shall provide three specialized collections: `text_chunks`, `code_blocks`, and `combined`.

- **FR-05: Query Processing Pipeline**

- The system shall classify user query intent (code_request, explanation, analysis, refactor, comparison).
- The system shall route queries to appropriate ChromaDB collections based on intent.
- The system shall expand queries with synonyms and variations for improved retrieval.
- The system shall retrieve top-k relevant documents (configurable, default k=5).
- The system shall rerank retrieved documents for diversity and relevance.

- **FR-06: Context Assembly and Citation**

- The system shall assemble context from retrieved documents within token limits (max 4000 tokens).
- The system shall track source documents and generate citation mappings.
- The system shall format citations with source URLs and tier information.
- The system shall filter duplicate or near-duplicate content (similarity threshold 0.85).
- The system shall include metadata (difficulty level, topic, source type) in context.

- **FR-07: LLM Integration**

- The system shall support Ollama for local open-source LLM inference as primary provider.
- The system shall use unified prompt templates for consistent response formatting.
- The system shall generate responses with inline citations [1], [2], [3].
- The system shall validate responses for citation presence and code syntax.
- The system shall provide configurable temperature and max_tokens parameters.
- The system shall support multiple open-source models (Llama, CodeLlama, Phi, etc.).

- **FR-08: CAST Chunking Feature**

- The system shall provide context-aware semantic text chunking for code and documentation.
- The system shall support fixed-size chunking for comparison purposes.
- The system shall visualize chunk boundaries and overlap in real-time.
- The system shall generate RAG prompts automatically from chunked content.
- The system shall display statistical comparison between chunking strategies.
- The system shall allow export of chunks and prompts in JSON format.
- The system shall support configurable chunk size and overlap parameters.

- **FR-09: Session Management**

- The system shall create unique session IDs for each user interaction.
- The system shall maintain conversation history within sessions.
- The system shall support multi-turn conversations with context preservation.
- The system shall implement session timeout (configurable, default 30 minutes).
- The system shall store session logs for debugging and analysis.

- **FR-10: Export and Download**

- The system shall allow copying analysis results to clipboard.
- The system shall support downloading results in JSON format with metadata.
- The system shall include timestamp and version information in exports.
- The system shall provide visual feedback for successful copy/download operations.

Phase 2 - Planned Features (Next Semester):

- **FR-11: Natural Language Code Generation**

- The system shall generate Python code from natural language descriptions.
- The system shall validate generated code syntax and provide compilation checking.
- The system shall incorporate best practices during code generation.
- The system shall provide explanatory comments in generated code.

- **FR-12: User Authentication**

- The system shall implement user registration and login functionality.
- The system shall store user credentials securely with encryption.
- The system shall maintain persistent user sessions across devices.
- The system shall provide password reset and account management features.

- **FR-13: Cloud Deployment**

- The system shall be deployed on cloud infrastructure (Azure or similar).
- The system shall support horizontal scaling for multiple concurrent users.
- The system shall implement load balancing for backend services.
- The system shall provide API endpoints for programmatic access.

3.2 Non-Functional Requirements

- **Performance Requirements:**

- Client-side code analysis completes within 2 seconds for files under 500 lines.
- RAG query processing provides initial response within reasonable time based on LLM inference speed.
- Vector database retrieval returns results within 100ms for semantic search queries.
- The web application loads initial page within 3 seconds on standard broadband connection.
- CAST chunking visualization update in real-time as parameters change.

- **Reliability and Availability:**

- The system gracefully handles invalid Python code with appropriate error messages.
- The system implements fallback mechanisms when primary LLM provider is unavailable.

- The system maintains data integrity during session management and storage operations.
- **Scalability Requirements:**
 - The vector database had the current knowledge base of 2,319 embeddings and can handle up to 10000 embeddings.
 - The system architecture is set to support future expansion of the knowledge base.
 - The backend is designed to support horizontal scaling when deployed to cloud.
- **Usability and Accessibility:**
 - The interface is intuitive and require minimal learning curve for Python developers.
 - The system provides clear error messages and guidance for failed operations.
 - Code editor supports standard keyboard shortcuts and accessibility features.
 - Results visualization is responsive and accessible across different screen sizes.
- **Compatibility:**
 - The system supports modern browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+.
 - The system functions on Windows, macOS, and Linux operating systems.
 - The system is compatible with Python 3.10+ for backend services.
 - The system supports Node.js 18+ for frontend development.

3.3 System Models

3.3.1 Data Flow Diagrams (DFD)

The Data Flow Diagrams illustrate how information flows through the SudhaarAI system at different levels of abstraction. The Level 0 DFD provides a high-level overview showing the main system processes and external entities, while the Level 1 DFD decomposes these processes into more detailed sub-processes, revealing the internal workings of code analysis, RAG retrieval, and LLM integration components.

Data Flow Diagram Level 0

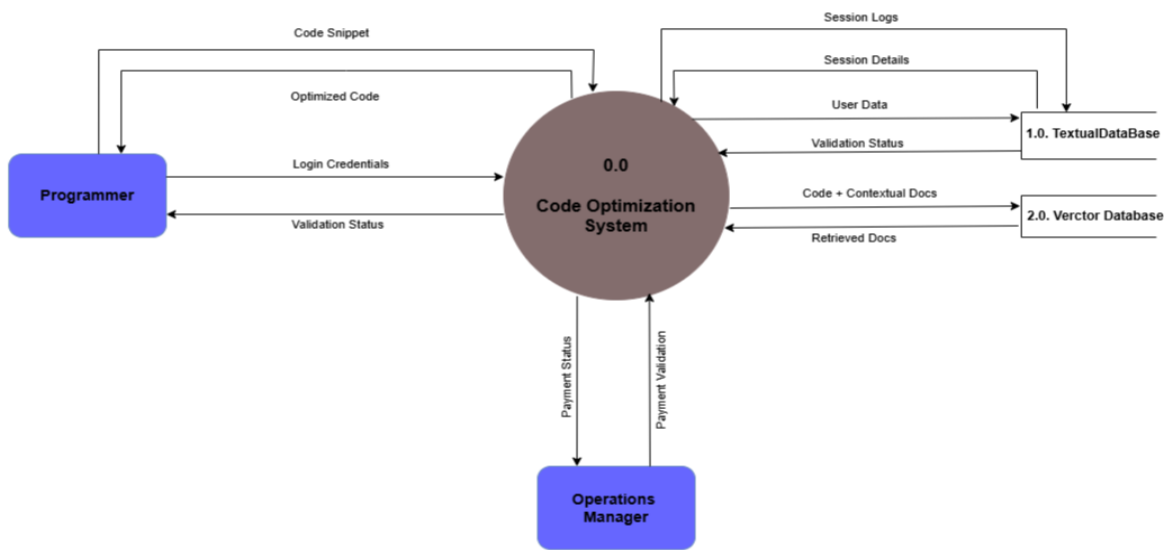


Figure 1: Data Flow Diagram Level 0

Data Flow Diagram Level 1



Figure 2: Data Flow Diagram Level 1

3.3.2 Use Case Diagrams

The Use Case Diagram depicts the primary interactions between different user types (students, professional developers, educators) and the SudhaarAI system. It illustrates key functionalities such as code analysis, RAG query submission, CAST chunking experimentation, and results visualization, showing how different actors engage with the system to achieve their goals.



Figure 3: Use Case Diagram

3.3.3 ER Diagram

The Entity-Relationship Diagram models the logical data structure and relationships within the SudhaarAI system. It shows how entities such as Users, Sessions, Documents, Chunks, and Embeddings relate to each other, with the physical implementation utilizing a hybrid approach where user and session data are stored relationally while document chunks and embeddings reside in ChromaDB vector database for optimized similarity search.

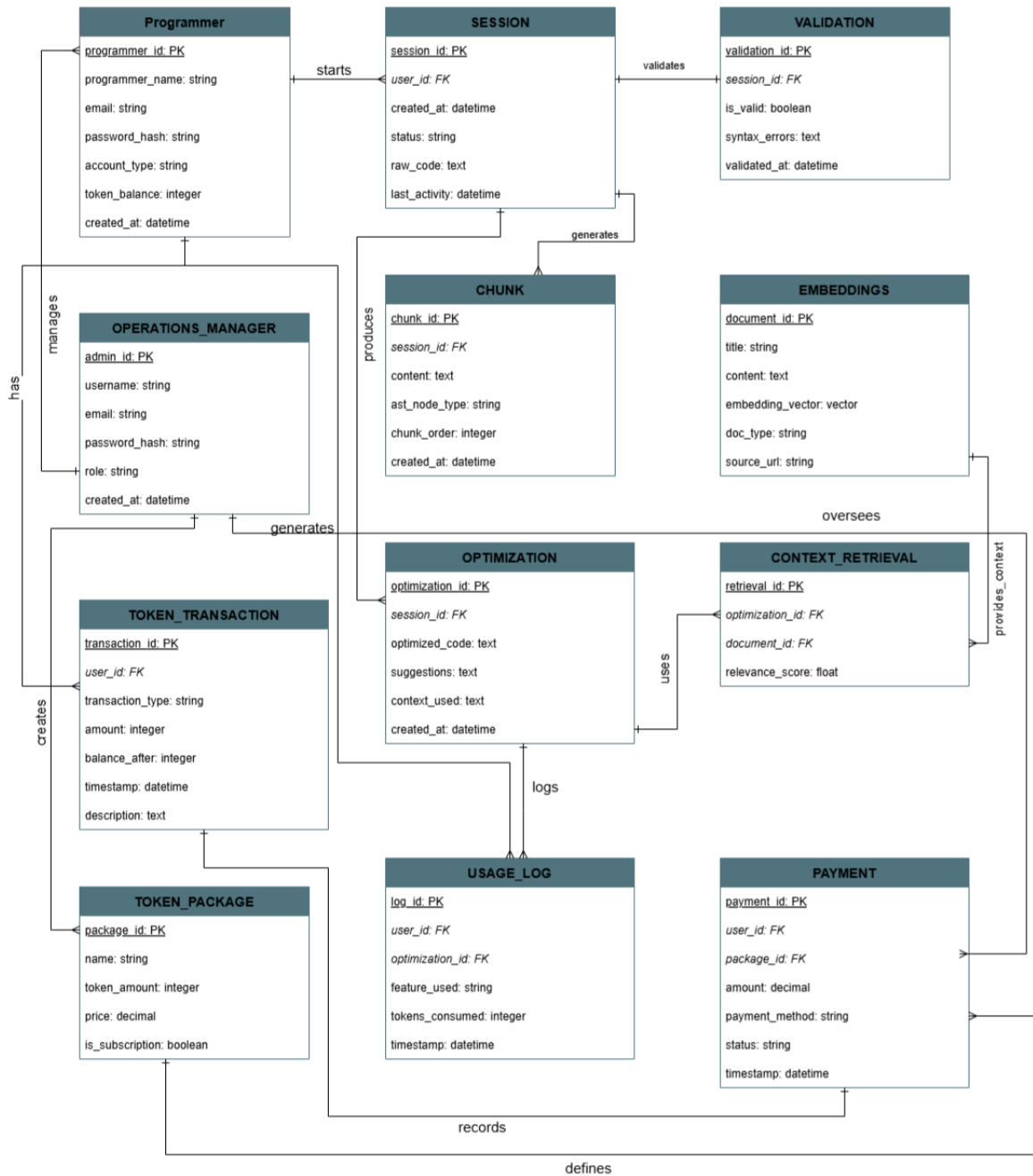


Figure 4: Entity Relationship Diagram

Note on Data Storage: While this ER diagram models the logical relationships and metadata dependencies between entities (e.g., matching a Document to its Chunks), the physical implementation utilizes a hybrid approach. User and Session data are typically stored in a relational structure, whereas Document Chunks and Embeddings are stored in **ChromaDB** (a vector database) optimized for high-dimensional similarity search.

3.3.4 Sequence And Activity Diagrams

The Sequence Diagram illustrates the temporal flow of interactions between system components during a typical code analysis and optimization workflow. It shows the step-by-step communication between the user interface, frontend analysis engine (custom JavaScript parser), backend RAG pipeline, vector database (ChromaDB), and LLM providers, demonstrating how requests are processed and responses are generated through the two-stage architecture.

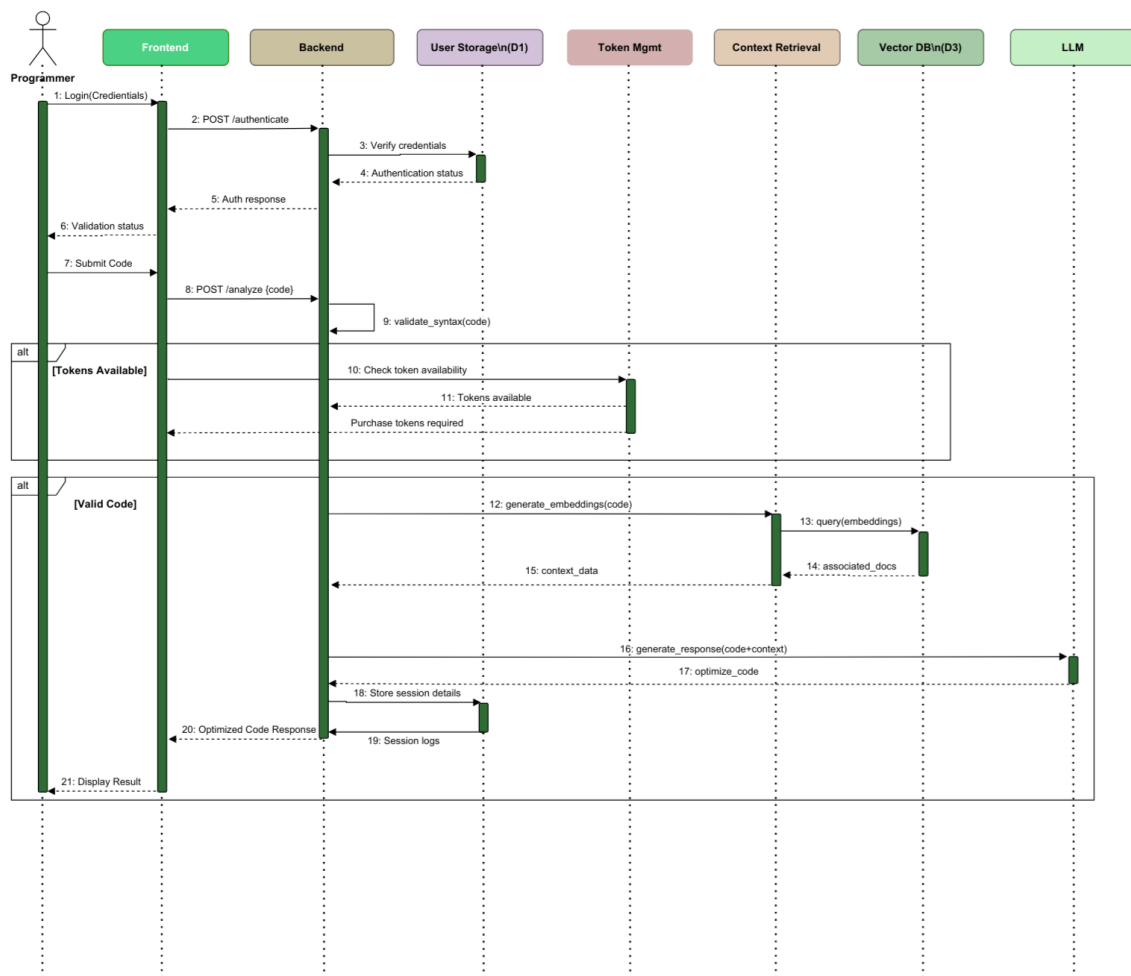


Figure 5: Sequence Diagram

3.4 External Interface Requirements

3.4.1 User Interfaces

- **Web Application Interface:** A responsive single-page application (SPA) built with React providing immediate code analysis and optimization suggestions.
- **Main Application Components:**
 - **Header Section:** Application branding, action buttons (Load Example, Clear, Copy Results, Download JSON), and visual feedback indicators.
 - **Drag-and-Drop Zone:** Visual overlay for file upload with drag-and-drop support and file validation feedback.
 - **Analysis Button:** Prominent button to trigger code analysis with loading state and progress indicators.
 - **Results Dashboard:** Comprehensive visualization including score gauge, metric cards, and detailed analysis tabs.
 - **CAST Chunking Tool:** Interactive interface for comparing chunking strategies with real-time visualization and statistics.
 - **Modal Dialogs:** Detailed metric explanations and methodology descriptions in overlay windows.
- **Responsive Design:** Mobile-friendly layout adapting to different screen sizes (desktop, tablet, mobile).
- **Visual Feedback:** Color-coded indicators (red/yellow/green), animated transitions, and toast notifications for user actions.
- **Accessibility Features:** Keyboard navigation support, ARIA labels, and semantic HTML for screen readers.

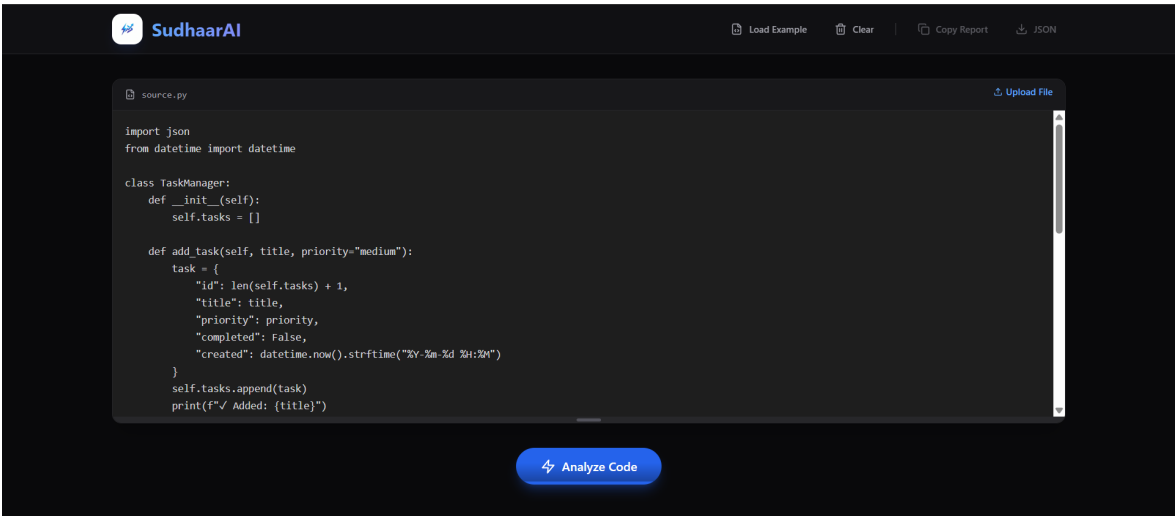


Figure 6: Main Application Interface

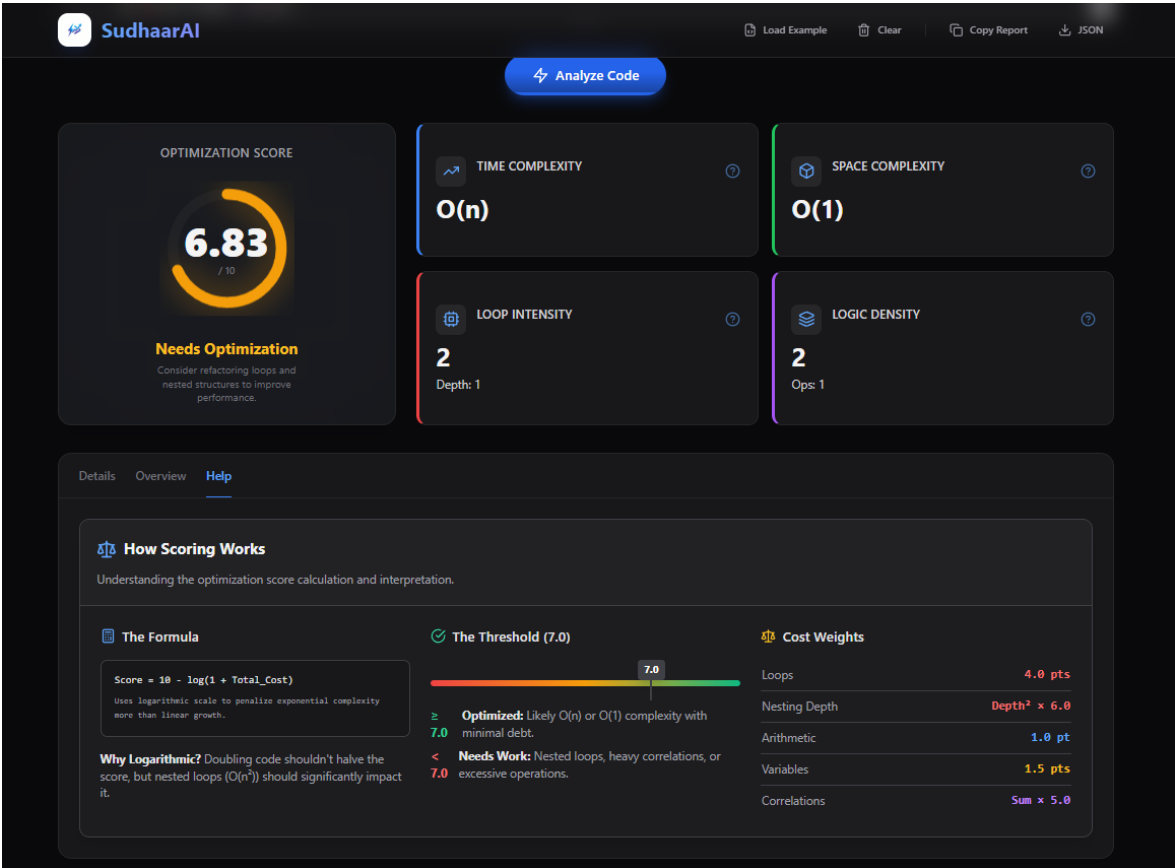


Figure 7: Analysis Results Dashboard

SudhaarAI
Load Example
Clear
Copy Report
JSON

CAST

Chunks: 2
Breaks: 0
Avg: 74

Fixed

Chunks: 2
Breaks: 1
Avg: 459

AI RAG Query (Generated)

Ask AI Expert
 Copy

Analyze this Python code's performance based on these metrics:

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$
- Score: 6.83/10
- Features: 2 loops (max depth 1), 2 conditionals.

Suggest specific refactoring techniques to improve O -notation efficiency and reduce cognitive complexity. Focus on the most expensive operations identified.

AI Expert Recommendation

```

=== RESPONSE ===

### Quick Definition:
The provided Python code has a time complexity of  $O(n)$  and a space complexity of  $O(1)$ . It contains two loops with a maximum depth of one and two conditional statements. The score is 6.83/10, indicating it could be optimized for better cognitive complexity.

### Core Concept:
To analyze and refactor the Python code based on the given metrics, we need to focus on optimizing the time complexity and reducing cognitive complexity. Since the current time complexity is  $O(n)$ , any improvements should aim to reduce the number of iterations or operations within loops. Additionally, simplifying conditional statements and reducing nested structures can help in lowering cognitive complexity [1].

### Simple Example:
Let's consider a simple example where we have a list and we want to find the maximum element.
'''python
# Basic code to find the maximum element in a list
def find_max(lst):
    max_element = lst[0]
    for num in lst:
        if num > max_element:
            max_element = num
    return max_element

# Explanation: This function iterates through the list once, making it  $O(n)$  time complexity.
'''

```

Figure 8: AI Expert Recommendations

Generated Code
 Apply to Editor

```

import json
from datetime import datetime

class TaskManager:
    def __init__(self):
        self.tasks = []

    def add_task(self, title, priority="medium"):
        task_id = len(self.tasks) + 1
        task = {
            "id": task_id,
            "title": title,
            "priority": priority,
            "completed": False,
            "created_at": datetime.now()
        }
        self.tasks.append(task)

    def complete_task(self, task_id):
        for task in self.tasks:
            if task["id"] == task_id:
                task["completed"] = True
                break

    def list_tasks(self):
        return json.dumps([task for task in self.tasks], indent=4)

```

Sources: 771c2ad3f683b753, ca18b5143735a822, 3691d8b71334e047

Figure 9: Generated Code View

3.4.2 Hardware Interfaces

- **Client Device:** Standard computer or mobile device with modern web browser; no special hardware required.
- **Server Hardware:** Standard server hardware for backend services; GPU optional for accelerated LLM inference.
- **Network Interface:** Standard internet connection for backend communication; client-side analysis works offline.
- **Storage:** Browser local storage for temporary session data; server storage for knowledge base and vector database.

3.4.3 Software Interfaces

- **Custom JavaScript Parser:** Custom parser for client-side code analysis and metrics calculation.
- **ChromaDB Interface:** Vector database API for semantic search and embedding storage.
- **Ollama API:** Local open-source LLM inference API for generating optimization suggestions.
- **sentence-transformers:** Python library for generating text embeddings.
- **Flask/FastAPI:** Backend framework providing REST API endpoints for RAG queries.
- **React Router:** Client-side routing for single-page application navigation.

3.4.4 Communication Interfaces

- **HTTP/HTTPS:** RESTful API communication between frontend and backend using JSON format.
- **WebSocket (Future):** Real-time streaming for LLM responses and live updates.

- **Data Formats:** JSON for API requests/responses, Python pickle for embeddings, SQLite for session storage.
- **API Endpoints:**
 - POST /api/rag - Submit RAG query and receive optimization suggestions
 - GET /health - Health check endpoint for backend status
 - Future: POST /api/generate - Natural language code generation
 - Future: POST /api/auth/login - User authentication
- **Error Handling:** Standardized error responses with HTTP status codes and descriptive messages.
- **CORS:** Cross-Origin Resource Sharing enabled for frontend-backend communication.

3.5 Constraints And Limitations

- **Language Limitation:** Currently supports Python only; multi-language support requires significant additional development.
- **Browser Dependency:** Requires modern browser with WebAssembly support; older browsers not supported.
- **Code Size Constraints:** Client-side analysis optimal for files under 1000 lines; larger files may experience performance issues.
- **LLM Inference Speed:** Response time depends on local hardware capabilities and selected LLM model.
- **Knowledge Base Currency:** Offline knowledge base requires manual updates; not automatically synchronized with latest Python releases.
- **Network Dependency:** Backend RAG features require server connectivity; only frontend analysis works offline.
- **Deployment Status:** Currently local development only; cloud deployment requires additional infrastructure setup.

- **Authentication:** No user authentication in current phase; planned for future implementation.

3.6 Assumptions And Dependencies

- **Assumption:** Users have modern web browsers supporting WebAssembly and ES6+ JavaScript.
- **Assumption:** Users are working with Python code and seek to improve code quality.
- **Assumption:** The curated knowledge base contains representative Python best practices.
- **Assumption:** Users have basic understanding of Python programming concepts.
- **Assumption:** Local development environment has sufficient resources for LLM inference.
- **Dependency:** Custom JavaScript parser for client-side code analysis.
- **Dependency:** Ollama for local open-source LLM inference.
- **Dependency:** ChromaDB for vector database functionality.
- **Dependency:** sentence-transformers for embedding generation.
- **Dependency:** React and Vite for frontend development.
- **Dependency:** Flask/FastAPI for backend API services.
- **Future Dependency:** Cloud infrastructure (Azure) for production deployment.
- **Future Dependency:** Authentication system for user management.

Chapter 4: Remaining Work And Timeline

This section outlines the roadmap for the completion of SudhaarAI, detailing the development phases, current progress, and timeline for delivering a fully functional AI-powered code analysis and optimization system.

4.1 Development Methodology And Architecture

The project follows an Agile iterative approach with regular sprints, enabling continuous testing and refinement of individual components before full integration. The development is structured around two major phases, with Phase 1 currently 40% complete.

Architecture Overview:

The system employs a two-stage architecture with the following key components:

- **Frontend (Stage 1):** React-based web application with custom JavaScript parser for client-side Python analysis
- **Backend (Stage 2):** Flask/FastAPI server with RAG pipeline for AI-powered recommendations
- **RAG System:** ChromaDB vector database with sentence-transformers embeddings
- **LLM Integration:** Ollama (local) for response generation using open-source models
- **Knowledge Base:** Offline curated collection of 500+ Python best practices
- **CAST Chunking:** Context-aware semantic text chunking tool for RAG optimization

4.2 Current Implementation Status (Phase 1)

Completed Components:

Table 1: Phase 1 Completed Work

Component	Status
Frontend Web Application	✓ React app with Monaco Editor, drag-and-drop, responsive design
Client-Side Analysis	✓ Custom JavaScript parser integration, basic complexity metrics, loop/conditional analysis
Results Visualization	✓ Score gauges, metric cards, tabbed interface, modal dialogs
RAG Knowledge Base	✓ 2,319 embeddings, ChromaDB with 3 collections, offline operation
Query Processing	✓ Intent classification, query routing, query rewriting
Context Assembly	✓ Document reranking, citation tracking, diversity filtering
LLM Integration	✓ Ollama provider, unified prompts, response validation
CAST Chunking Tool	✓ Context-aware chunking, visualization, comparison statistics
Session Management	✓ Session tracking, conversation history, timeout handling
Export Functionality	✓ Copy to clipboard, JSON download with metadata

4.3 Remaining Work (Phase 2 - Next Semester)

Planned Development:

Table 2: Phase 2 Planned Work

Component	Description
Code Generation	Natural language to Python code generation with best practices
User Authentication	Registration, login, password management, session persistence
Cloud Deployment	Azure deployment with scalability and load balancing
Multi-Language Support	Potential expansion beyond Python (feasibility dependent)
Advanced Features	Enhanced session management, user preferences, history tracking

Detailed Task Status And Timeline:

Table 3: Outstanding Tasks With Status And Deadlines

Task	Status	Priority	Expected Completion	Assigned To
Code Generation Feature	Pending	High	March 2026	Abu Bakar
User Authentication System	Pending	High	March 2026	Raqib Hayat
Cloud Deployment Setup	Pending	Medium	May 2026	Both
Advanced RAG Techniques	In Progress	Medium	April 2026	Abu Bakar
Performance Optimization	Pending	Low	May 2026	Both
User Testing & Feedback	Pending	High	April 2026	Both
Documentation & Dissertation	In Progress	High	May 2026	Both
Final Integration & Testing	Pending	High	May 2026	Both

4.3.1 Individual Task Distribution

Abu Bakar - AI/ML and Backend Components

Table 4: Abu Bakar Task Timeline

Period	Activities
Completed	• RAG knowledge base curation • Vector database setup • Embedding generation • Query processing pipeline • LLM integration • Context assembly
Next Semester	• Code generation feature • Advanced RAG techniques • Model fine-tuning • Performance optimization • Dissertation writing (AI/ML components)

Table 5: Raqib Hayat Task Timeline

Period	Activities
Completed	• React frontend development • Custom JavaScript parser integration • Custom code editor setup • Results visualization • CAST chunking tool • Export functionality • Responsive design
Next Semester	• User authentication system • Cloud deployment (Azure) • Frontend-backend integration enhancements • User testing • Technical documentation

Table 6: Collaborative Timeline

Period	Activities
Current Semester	• Weekly progress meetings • Code reviews • Architecture discussions • Integration testing • Bug fixes
Next Semester	• User studies • Performance benchmarking • Final integration • Deployment • Dissertation preparation & presentation

Raqib Hayat - Frontend and System Integration

Collaborative Activities

4.4 Success Criteria And Evaluation

Primary Success Metrics:

- **Functionality:** All planned features implemented and working correctly
- **Code Quality:** System provides accurate and relevant optimization suggestions
- **User Experience:** Intuitive interface with responsive feedback
- **Performance:** Acceptable response times for analysis and RAG queries
- **Citation Accuracy:** All recommendations properly attributed to authoritative sources

Verification Methods:

- User testing with students and professional developers

- Code quality assessment on benchmark Python projects
- Performance profiling under various load conditions
- Comparison with existing code analysis tools
- Demonstration of RAG optimization through CAST chunking

Technology Stack Implementation

Frontend Technologies:

- **Framework:** React 18+ with Vite for fast development and optimized builds
- **Code Editor:** Custom code editor for professional code editing with syntax highlighting
- **Client-Side Analysis:** Custom JavaScript parser for code analysis and metrics calculation
- **UI Libraries:** Tailwind CSS for styling, Framer Motion for animations
- **State Management:** React Hooks (useState, useEffect, custom hooks)
- **Build Tool:** Vite with hot module replacement for rapid development
- **Package Manager:** npm or pnpm for JavaScript dependencies

Backend Technologies:

- **Framework:** Flask or FastAPI for REST API development
- **Python Version:** Python 3.10+ with type hints
- **Package Manager:** uv for fast, reliable Python dependency management
- **Vector Database:** ChromaDB for storing and querying embeddings
- **Embeddings:** sentence-transformers (all-MiniLM-L6-v2) for semantic search
- **LLM Providers:** Ollama (local inference) for open-source model support

- **Code Analysis:** Python AST module, Radon for complexity metrics
- **Session Storage:** SQLite for lightweight session management

AI/ML Stack:

- **Embedding Model:** sentence-transformers/all-MiniLM-L6-v2 (384 dimensions)
- **Vector Search:** ChromaDB with cosine similarity
- **LLM Models:** Llama 3.1, CodeLlama, Phi-3 (via Ollama)
- **Prompt Engineering:** Unified prompt templates with structured response format
- **Response Validation:** Citation checking, code syntax validation

Development Tools:

- **Version Control:** Git for source code management
- **Code Quality:** ESLint (JavaScript), Ruff (Python), Prettier (formatting)
- **Python Tools:** uv for package management, Radon for complexity analysis
- **API Testing:** Postman or curl for backend endpoint testing
- **Browser DevTools:** Chrome/Firefox DevTools for frontend debugging

Deployment Stack:

- **Frontend Hosting:** Vercel, Netlify, or Azure Static Web Apps
- **Backend Hosting:** Azure Container Instances or App Service
- **Database:** Managed ChromaDB instance or self-hosted container
- **CI/CD:** GitHub Actions for automated testing and deployment
- **Monitoring:** Application Insights or similar for performance tracking

References

- [1] Python Software Foundation, "PEP 0 – Index of Python Enhancement Proposals (PEPs)," Python.org. [Online]. Available: <https://www.python.org/dev/peps/>. [Accessed: Jan. 19, 2026].
- [2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.
- [3] "ChromaDB Documentation," Chroma. [Online]. Available: <https://docs.trychroma.com/>. [Accessed: Jan. 19, 2026].
- [4] "Sentence-Transformers Documentation," SBERT.net. [Online]. Available: <https://www.sbert.net/>. [Accessed: Jan. 19, 2026].
- [5] "Pyodide Documentation," Pyodide. [Online]. Available: <https://pyodide.org/>. [Accessed: Jan. 19, 2026].
- [6] "Monaco Editor Documentation," Microsoft. [Online]. Available: <https://microsoft.github.io/monaco-editor/>. [Accessed: Jan. 19, 2026].
- [7] "Ollama Documentation," Ollama. [Online]. Available: <https://ollama.ai/>. [Accessed: Jan. 19, 2026].
- [8] "React Documentation," Meta. [Online]. Available: <https://react.dev/>. [Accessed: Jan. 19, 2026].
- [9] "FastAPI Documentation," FastAPI. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Jan. 19, 2026].
- [10] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308-320, Dec. 1976.
- [11] "Real Python Tutorials," Real Python. [Online]. Available: <https://realpython.com/>. [Accessed: Jan. 19, 2026].
- [12] "The Hitchhiker's Guide to Python," Python Guide. [Online]. Available: <https://docs.python-guide.org/>. [Accessed: Jan. 19, 2026].

APPENDICES

Appendix A: Project Dependencies And Setup

A.1 Project Dependencies

Frontend Dependencies (package.json)

Listing 1: *Frontend Package Configuration*

```
1 {
2   "name": "sudhaar-ai",
3   "version": "0.0.0",
4   "type": "module",
5   "dependencies": {
6     "@monaco-editor/react": "^4.7.0",
7     "clsx": "^2.1.1",
8     "framer-motion": "^12.23.26",
9     "lucide-react": "^0.344.0",
10    "react": "^18.2.0",
11    "react-dom": "^18.2.0",
12    "tailwind-merge": "^3.4.0"
13  },
14  "devDependencies": {
15    "@testing-library/jest-dom": "^6.9.1",
16    "@testing-library/react": "^16.3.1",
17    "@testing-library/user-event": "^14.6.1",
18    "@vitejs/plugin-react": "^4.2.1",
19    "autoprefixer": "^10.4.18",
20    "eslint": "^8.57.0",
21    "postcss": "^8.4.35",
22    "tailwindcss": "^3.4.1",
23    "vite": "^5.1.6",
24    "vitest": "^4.0.16"
25  }
26 }
```

Backend Dependencies (requirements.txt)

Listing 2: *Backend Python Dependencies*

```
1 beautifulsoup4
2 pyyaml
3 numpy
4 tqdm
5 sentence-transformers
6 chromadb
7 requests
8 transformers
9 torch
10 accelerate
11 flask
12 flask-cors
```

A.2 Installation And Setup

Frontend Setup

Listing 3: *Frontend Installation Steps*

```
1 # Navigate to frontend directory
2 cd "Code_Optimizer_Frontend"
3
4 # Install dependencies
5 npm install
6
7 # Start development server
8 npm run dev
9
10 # Build for production
11 npm run build
12
13 # Run tests
14 npm run test
```


Backend Setup

Listing 4: *Backend Installation Steps*

```
1 # Navigate to backend directory
2 cd "RAG_Backend"
3
4 # Create virtual environment using uv
5 uv venv
6
7 # Install dependencies
8 uv pip install -r requirements.txt
9
10 # Set up environment variables
11 cp .env.example .env
12 # Edit .env with your API keys
13
14 # Run Phase 2 embedding pipeline
15 cd "Phase_2_-_Embedding_Generation_and_Vector_Database_Setup"
16 uv run python run_embedding_pipeline.py
17
18 # Run Phase 3 RAG pipeline
19 cd "../Phase_3_-_RAG_System_Integration"
20 uv run python run_rag_pipeline.py "Your_query_here"
```

System Requirements

Frontend Requirements:

- Node.js 18+ and npm/pnpm
- Modern browser with JavaScript ES6+ support
- Minimum 4 GB RAM (8 GB recommended)
- Display resolution 1366x768 or higher

Backend Requirements:

- Python 3.10+
- Minimum 16 GB RAM (32 GB recommended for LLM)
- 10 GB free disk space
- Optional: CUDA-compatible GPU for accelerated inference

Appendix B: System Overview And Validation

B.1 System Architecture

Stage 1: Client-Side Analysis (Frontend)

- Custom JavaScript parser for Python code analysis
- Custom code editor for code input
- Real-time metrics calculation
- No server communication required
- Privacy-preserving design

Stage 2: RAG-Powered Optimization (Backend)

- Query processing pipeline
- ChromaDB vector database
- Semantic retrieval system
- LLM integration (Ollama)
- Citation-backed recommendations

RAG Pipeline Flow:

1. **Query Processing:** Intent classification, routing, query rewriting
2. **Retrieval:** Semantic search across 2,319 embeddings
3. **Reranking:** Diversity filtering and relevance scoring
4. **Context Assembly:** Build LLM context with citations
5. **LLM Generation:** Generate response with inline citations
6. **Validation:** Check citation presence and code syntax
7. **Response Formatting:** Format final answer with metadata

B.2 System Validation

Frontend Test Suite Results:

Table 7: Frontend Test Suite Results

Test Suite	Status
Accessibility Tests	PASS
Copy/Download Functionality	PASS
Drag And Drop File Upload	PASS
Error Handling	PASS
Code Analysis Integration	PASS

Retrieval System Performance Metrics:

Table 8: Retrieval System Performance Metrics

Metric	Score	Status
Top-1 Precision	90%	PASS
Top-5 Recall	100%	PASS
Mean Reciprocal Rank (MRR)	0.94	PASS
Average Latency	30.57 ms	PASS

Table 9: Dataset Collection Statistics

Tier	Category	Sources	Size
Tier 1	Official PEPs	6	550 KB
Tier 2	Community Articles	26	3.8 MB
Tier 2	GitHub Repositories	1	Variable
Tier 3	Stack Overflow Q&A	100	2.1 MB
Tier 3	Python Documentation	18	3.0 MB
Total	All Sources	151	7.6 MB

Appendix C: Dataset Statistics

C.1 Knowledge Base Summary

C.2 Processing Pipeline Results

Table 10: Phase 1 And Phase 2 Processing Metrics

Metric	Value
Parsed JSON Files	100+
Extracted Code Blocks	1,410
Valid Code Blocks	983 (69.7%)
Text Chunks Generated	909
Total Embeddings	2,319
Vector Dimensions	384
ChromaDB Collections	3
Average Search Latency	30.57 ms
Top-1 Precision	90%
Top-5 Recall	100%
Processing Time (Full Pipeline)	6 minutes

C.3 Content Coverage By Category

Appendix D: Future Work

D.1 Planned Features (Phase 2)

- **Natural Language Code Generation:** Generate Python code from descriptions with best practices and comments.

Table 11: Best Practice Category Coverage

Category	Sources	Tiers Covered
Style/Formatting	12	1, 2, 3
Docstrings	5	1, 2, 3
Type Hints	4	1, 2, 3
Error Handling	8	2, 3
Testing	7	2, 3
Logging	8	1, 2, 3
Project Structure	5	2
Generators/Iterators	4	2, 3
Decorators	3	2, 3
Context Managers	3	2, 3

- **User Authentication:** Secure registration, login, and session management.
- **Cloud Deployment:** Azure setup, horizontal scaling, and load balancing.

D.2 Research Directions

- Advanced RAG techniques (cross-encoder reranking)
- Multi-language support beyond Python
- Real-time collaborative code optimization
- Integration with CI/CD pipelines
- Performance prediction models
- Enhanced web-based collaboration features

Appendix E: Glossary

AST (Abstract Syntax Tree) Tree representation of source code structure

CAST Chunking Context-Aware Semantic Text chunking for RAG optimization

ChromaDB Open-source vector database for embeddings

Cyclomatic Complexity Metric measuring code complexity via independent paths

Embedding Numerical vector representation of text or code

LLM (Large Language Model) AI model trained on vast text data

Monaco Editor Custom code editor component

PEP (Python Enhancement Proposal) Official Python design documents

Custom JavaScript Parser Custom parser for client-side Python code analysis

RAG (Retrieval-Augmented Generation) AI technique combining retrieval with generation

Semantic Search Search based on meaning rather than keywords

Vector Database Database optimized for high-dimensional vector similarity search

WebAssembly Binary instruction format for web browsers

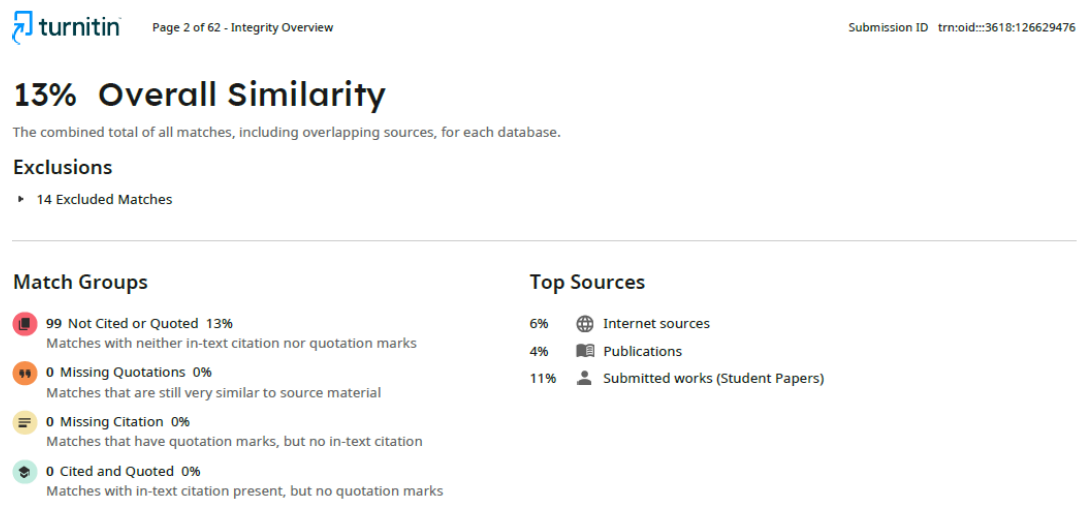


Figure 10: Turnitin Report